HaikyuuHeights

Complete Software Architecture and Codebase Documentation

Mentor review handoff document

Generated: 27 October 2026
Source scope: client/src, server/src, server/scripts
Analyzed: 136 source files and approximately 16,607 lines
Excluded: node_modules, generated dist bundles, local secrets, and runtime logs

This document describes the implementation currently present in the workspace. It is intended to be supplied to another AI or mentor as a complete technical handoff. The documentation is code-derived rather than based only on the README, so it distinguishes implemented behavior from planned or claimed features.

1. Executive Summary

HaikyuuHeights is a broker-free real-estate marketplace implemented as a MERN-style application. The React/Vite client provides public property discovery and role-specific dashboards for buyers, sellers, and administrators. The Express backend exposes REST endpoints, applies authentication and role middleware, persists domain data in MongoDB through Mongoose, stores images in Cloudinary, and uses Supabase only as the Google OAuth identity provider. Application sessions are represented by JWTs stored in browser localStorage.

Primary business capabilities: property listings, search and filters, visit bookings, favorites, buyer-seller conversations and messages, reviews, seller verification, commissions, and administrative moderation.

Important architectural boundary: Supabase authenticates Google users, but MongoDB remains the application database. A successful Google redirect alone is not sufficient; the backend must validate the Supabase access token and then find or create the corresponding MongoDB User document.

2. Architecture at a Glance

The system is a two-process web application. The browser talks to the Express API over HTTP. The API talks to MongoDB Atlas for records, Cloudinary for media, and Supabase Auth for Google identity validation.

```
Browser
  React 18 + Vite + React Router + Redux Toolkit + Axios
       |
       | REST /api/*, JWT Authorization header, credentials enabled
       v
Express API (Node.js)
  CORS -> Helmet -> body parsers -> rate limits -> routes
  Routes -> validators -> auth/role/verification middleware -> controllers
       |                |                  |
       v                v                  v
  MongoDB/Mongoose    Cloudinary         Supabase Auth
  users, properties,  avatar/property    Google token validation
  bookings, reviews,  and document media
  messages, favorites,
  verification, commissions
```

3. Repository Structure

Area	Responsibility	Evidence
client/src/pages	Public, buyer, seller, and admin screens	29 JSX pages
client/src/components	Reusable UI and domain widgets	15 JSX components
client/src/features	Redux Toolkit state slices and async thunks	6 feature files
client/src/services	Axios-backed API service modules	11 service modules
client/src/routes	Public, protected, and role-based routing	AppRoutes, ProtectedRoute, RoleRoute
server/src/routes	HTTP endpoint declarations	11 route modules
server/src/controllers	Request orchestration and business operations	11 controllers
server/src/models	Mongoose schemas and persistence rules	10 models
server/src/middleware	Security, auth, role, upload, validation, errors	6 middleware modules
server/src/services	Cloudinary, commission, and review logic	3 services

4. Frontend Architecture

The client starts in main.jsx, creates a Redux Provider and BrowserRouter, then renders App. App delegates routing to AppRoutes. Public pages use PublicLayout; role dashboards use DashboardLayout. Redux slices own asynchronous API operations and persist authentication values under hh_token and hh_user.

Frontend layers

Layer	Implementation	Purpose
Bootstrap	main.jsx, App.jsx	Mount React, Redux, router, toaster, global CSS
Routing	routes/AppRoutes.jsx	Maps URLs to public and dashboard pages

Layer	Implementation	Purpose
Guards	ProtectedRoute.jsx, RoleRoute.jsx	Require JWT/user and enforce BUYER/SELLER/ADMIN roles
State	features/*, store/store.js	Async thunks, loading/error state, domain collections
API	services/api.js and *.service.js	Axios base URL, JWT header, endpoint wrappers
UI	components/*, layouts/*	Reusable visual and interaction patterns
Integration	lib/supabase.js	Browser Supabase client for Google OAuth redirect

Client routes

Route group	Examples	Access	
Public	/	/properties	/about
Auth	/login	/register	Unauthenticated
Buyer	/buyer/dashboard	/buyer/bookings	/buyer/favorites
Seller	/seller/dashboard	/seller/properties	/seller/verificatio
Admin	/admin/dashboard	/admin/users	/admin/verificati

5. Backend Architecture

server.js loads environment variables, validates required infrastructure settings, connects to MongoDB, and only then starts listening. app.js configures security headers, CORS, body parsing, rate limiting, health checking, API route mounting, 404 handling, and centralized error handling.

Request pipeline

```
Incoming request
  -> helmet / CORS / JSON or multipart parsing
  -> /api rate limiter
  -> route-level auth limiter where configured
  -> authMiddleware (JWT verification when protected)
  -> authorizeRoles (role gate)
  -> verifiedMiddleware (seller verification gate where required)
  -> express-validator arrays + validateRequest
  -> controller wrapped by asyncHandler
  -> ApiResponse on success or errorMiddleware on failure
```

Controllers use ApiError for expected failures and ApiResponse for consistent successful responses. asyncHandler forwards rejected promises to errorMiddleware. The error middleware maps Mongoose cast, duplicate-key, validation, and JWT errors to HTTP responses and logs auth failures through authLogger.

6. Authentication and Authorization

Password registration/login

Registration validates fullName, email, phone, password, confirmPassword, and optional role. The User model hashes passwords in an async pre-save hook. Login normalizes the email, selects the hidden password field, compares using bcrypt, rejects suspended users, signs a JWT containing userId, role, and email, and returns the user plus seller verification status where applicable.

Google/Supabase login

1. Login.jsx calls supabase.auth.signInWithOAuth({ provider: "google", redirectTo: window.location.origin + "/login" }).
2. The tab records hh_google_oauth_pending in sessionStorage.
3. After redirect, Login.jsx calls supabase.auth.getSession().
4. The access token is posted to POST /api/auth/supabase.
5. The server calls Supabase auth.getUser(accessToken).
6. It finds or creates a MongoDB User, rejects suspended users, signs the application's JWT, and returns application user data.
7. The client stores hh_token and hh_user, then signs out of the temporary Supabase browser session.

Operational dependency: the Google flow requires all of Supabase, the backend, MongoDB, and correct redirect/CORS configuration. Supabase configuration does not remove the need for MongoDB because the application's roles and domain records are stored there.

7. REST Endpoint Inventory

Method	Endpoint	Route module
GET	/api/admin/stats	server/src/routes/admin.routes.js
GET	/api/admin/recent-activity	server/src/routes/admin.routes.js
GET	/api/admin/properties	server/src/routes/admin.routes.js
PATCH	/api/admin/properties/:id/status	server/src/routes/admin.routes.js
POST	/api/auth/register	server/src/routes/auth.routes.js
POST	/api/auth/login	server/src/routes/auth.routes.js
POST	/api/auth/supabase	server/src/routes/auth.routes.js
GET	/api/auth/me	server/src/routes/auth.routes.js
PATCH	/api/auth/change-password	server/src/routes/auth.routes.js
PATCH	/api/auth/profile	server/src/routes/auth.routes.js
POST	/api/booking/	server/src/routes/booking.routes.js
GET	/api/booking/buyer/mine	server/src/routes/booking.routes.js
PATCH	/api/booking/:id/cancel	server/src/routes/booking.routes.js
GET	/api/booking/seller/mine	server/src/routes/booking.routes.js
PATCH	/api/booking/:id/accept	server/src/routes/booking.routes.js
PATCH	/api/booking/:id/reject	server/src/routes/booking.routes.js
PATCH	/api/booking/:id/complete	server/src/routes/booking.routes.js
GET	/api/booking/	server/src/routes/booking.routes.js
GET	/api/commission/seller/mine	server/src/routes/commission.routes.js
GET	/api/commission/buyer/mine	server/src/routes/commission.routes.js
GET	/api/commission/	server/src/routes/commission.routes.js
POST	/api/contact/	server/src/routes/contact.routes.js
POST	/api/conversation/	server/src/routes/conversation.routes.js
GET	/api/conversation/	server/src/routes/conversation.routes.js
GET	/api/conversation/:id/messages	server/src/routes/conversation.routes.js
POST	/api/conversation/:id/messages	server/src/routes/conversation.routes.js
GET	/api/favorite/	server/src/routes/favorite.routes.js
POST	/api/favorite/	server/src/routes/favorite.routes.js
GET	/api/favorite/check/:propertyId	server/src/routes/favorite.routes.js
DELETE	/api/favorite/:propertyId	server/src/routes/favorite.routes.js
GET	/api/property/	server/src/routes/property.routes.js
GET	/api/property/seller/mine	server/src/routes/property.routes.js
GET	/api/property/:id	server/src/routes/property.routes.js
POST	/api/property/	server/src/routes/property.routes.js
PATCH	/api/property/:id	server/src/routes/property.routes.js
DELETE	/api/property/:id	server/src/routes/property.routes.js
DELETE	/api/property/:id/images/:imageId	server/src/routes/property.routes.js
PATCH	/api/property/:id/images/:imageId/primary	server/src/routes/property.routes.js
GET	/api/review/property/:id	server/src/routes/review.routes.js
GET	/api/review/seller/:id	server/src/routes/review.routes.js
POST	/api/review/	server/src/routes/review.routes.js
GET	/api/review/my	server/src/routes/review.routes.js

Method	Endpoint	Route module
PATCH	/api/review/:id	server/src/routes/review.routes.js
DELETE	/api/review/:id	server/src/routes/review.routes.js
GET	/api/review/	server/src/routes/review.routes.js
POST	/api/sellerVerification/submit	server/src/routes/sellerVerification.routes.js
GET	/api/sellerVerification/status	server/src/routes/sellerVerification.routes.js
GET	/api/sellerVerification/	server/src/routes/sellerVerification.routes.js
GET	/api/sellerVerification/:id	server/src/routes/sellerVerification.routes.js
PATCH	/api/sellerVerification/:id/approve	server/src/routes/sellerVerification.routes.js
PATCH	/api/sellerVerification/:id/reject	server/src/routes/sellerVerification.routes.js
GET	/api/user/	server/src/routes/user.routes.js
GET	/api/user/:id	server/src/routes/user.routes.js
PATCH	/api/user/:id	server/src/routes/user.routes.js
PATCH	/api/user/:id/suspend	server/src/routes/user.routes.js
DELETE	/api/user/:id	server/src/routes/user.routes.js

8. Domain Models and Data Ownership

MongoDB is the system of record for application data. The following model inventory is derived from the Mongoose schema files. Field lists are the principal top-level schema fields; nested objects and validation rules remain in the corresponding model source.

Model	Principal fields
Booking	buyer, seller, property, requestedDate, requestedTime, message, status, sellerNote
Commission	property, seller, buyer, booking, transactionAmount, sellerCommissionRate, sellerCommissionAmount, buyerCommissionRate, buye
Contact	fullName, email, phone, subject, message, status
Conversation	property, buyer, seller, lastMessage, lastMessageAt
Favorite	buyer, property
Message	conversation, sender, content, isRead
Property	url, publicId, isPrimary, title, description, propertyType, listingType, price, city, state, address, latitude, longitude, bedrooms, bathroor
Review	reviewer, property, seller, rating, comment
SellerVerification	url, publicId, fileType, originalName, seller, identityProof, ownershipProof, status, adminNote, reviewedBy, reviewedAt
User	fullName, email, phone, password, role, avatar, isActive, isSuspended

Domain relationships

Relationship	Meaning
User -> Property	A seller owns/listed properties; buyer-facing property records contain seller references.
User -> Booking	A buyer schedules a visit; the property seller manages the request.
User -> Review	Users create property reviews; admins moderate them.
User -> Favorite	A buyer saves a property and can remove it later.
User -> Conversation -> Message	Buyer and seller communicate through a property-aware conversation.
User -> SellerVerification	A seller submits documents and an admin changes the verification status.
Booking -> Commission	Completed/eligible transactions feed commission tracking.

9. Feature Workflows

Workflow	Implementation flow
Property discovery	Public properties page calls property service; filters and search are represented in property state; cards and map components rende
Property publishing	Seller form submits multipart data through property service; upload middleware validates images; Cloudinary stores media; controlle
Booking	Buyer creates a visit booking; seller or admin changes status; buyer can view or cancel according to controller rules.
Favorites	Buyer adds/removes property references through favorite endpoints and Redux state updates the saved collection.
Messaging	Conversation endpoints create/retrieve threads and message endpoints send/read messages; buyer and seller dashboard pages pr
Reviews	Reviews are associated with properties/users, validated server-side, and can be moderated by admin endpoints.
Seller verification	Seller uploads required documents; admin reviews, approves/rejects, and protected seller operations can require approved verificati
Commissions	Commission service/controller calculate and expose seller/admin financial summaries and status tracking.

10. Configuration and Deployment

The client uses VITE_API_URL with a localhost fallback and has a Vite development proxy for /api. The server uses PORT, MONGODB_URI, JWT_SECRET, JWT_EXPIRES_IN, Cloudinary credentials, CLIENT_URL, NODE_ENV, and Supabase server credentials. Local .env files contain secrets and must never be committed or copied into a mentor-facing document.

Variable	Used by	Purpose
VITE_API_URL	Client	REST API base URL
VITE_SUPABASE_URL / VITE_SUPABASE_ANON_KEY	Client	Supabase browser OAuth client

Variable	Used by	Purpose
MONGODB_URI	Server	MongoDB Atlas connection
JWT_SECRET / JWT_EXPIRES_IN	Server	Application session signing and lifetime
SUPABASE_URL / SUPABASE_ANON_KEY	Server	Validate Google access tokens
CLOUDINARY_*	Server	Image and document media storage
CLIENT_URL	Server	CORS allow-list

Recommended deployment sequence

```
1. Provision MongoDB Atlas and allow the deployed server IP.
2. Provision Cloudinary and create upload credentials.
3. Configure Supabase Google provider and production redirect URL.
4. Set server environment variables in the hosting platform.
5. Start server; it must connect to MongoDB before listening.
6. Build client with production VITE_API_URL and Supabase values.
7. Configure CORS CLIENT_URL to the exact deployed client origin.
8. Verify HTTPS redirect URLs and never expose server secrets.
```

11. Security and Reliability Review

Area	Current implementation	Mentor discussion / improvement
Passwords	bcryptjs hashing, password excluded from normal queries/JSON	Add password reset and credential rotation policy.
JWT	Signed JWT in localStorage and Authorization header	For production, consider secure httpOnly cookies and refresh-token rotation to reduce X
Input validation	express-validator on auth and domain request paths	Ensure every mutating route has a validator and add schema-level limits.
Rate limiting	Global API limiter plus stricter auth limiter	Use a shared store when horizontally scaling beyond one process.
CORS	Explicit localhost origins and CLIENT_URL	Use exact production origins; do not broaden to wildcard with credentials.
Uploads	Multer/Cloudinary integration and size/type controls	Enforce content inspection, quotas, and malware scanning for production.
Errors	Centralized error middleware and auth event logging	Avoid returning development stack traces outside local development.
Secrets	Examples are separate from local env files	Rotate any credentials that have ever been shared or committed.
Tests	No test suite is present in the source inventory	Add unit, API integration, and end-to-end authentication tests.

12. Code-Derived Gaps and Clarifications

The repository's completion summaries describe the intended product broadly, but the source tree is the authoritative implementation. The following points should be stated honestly in a mentor presentation:

• The server cannot start until MongoDB Atlas is reachable; Supabase does not replace MongoDB.

• Google login has two phases: Supabase identity authentication followed by backend JWT/application-user creation.

• The application is REST-based; no websocket transport is visible in the source inventory, so messaging should be described as API-backed rather than claiming socket-level real time.

• The source contains no automated test directory or test script in the package manifests; validation should be presented as manual/development verification unless tests are added.

• The local environment files contain sensitive credentials and are intentionally excluded from this document.

• The current codebase includes a generated dist folder and operational logs; these are artifacts, not primary source architecture.

13. Complete Source Inventory

Every application source file below was included in the codebase analysis. The symbol column is automatically extracted from declarations and is a navigation aid; the file remains the definitive implementation. This appendix gives another AI a complete map before it reads individual files.

File	Lines	Detected declarations
client/src/App.jsx	17	App, dispatch, token
client/src/components/booking/BookingCard.jsx	168	BookingCard, buyer, canAccept, canCancel, canComplete, canReject, canReview, image, property, seller, status
client/src/components/common/Badge.jsx	22	Badge, cls, variants
client/src/components/common/DashboardCard.jsx	34	DashboardCard, colors
client/src/components/common/ErrorMessage.jsx	15	ErrorMessage
client/src/components/common/Footer.jsx	81	Footer
client/src/components/common/LoadingSpinner.jsx	24	LoadingSpinner, sizes, spinner
client/src/components/common/Modal.jsx	46	Modal, sizes
client/src/components/common/Navbar.jsx	209	Navbar, adminMenuItems, buyerMenuItems, getDashboardLink, handleLogout, isActive, location, menuItems, navLin
client/src/components/common/Pagination.jsx	64	Pagination, delta, i, left, pages, right
client/src/components/common/RatingStars.jsx	23	RatingStars, sizes
client/src/components/property/FilterPanel.jsx	150	FilterPanel, handle
client/src/components/property/MapView.jsx	52	MapView
client/src/components/property/PropertyCard.jsx	151	PropertyCard, dispatch, favoriteIds, handleFavoriteToggle, imageUrl, isAuthenticated, isBuyer, isFavorited, listingBad
client/src/components/property/SearchBar.jsx	31	SearchBar, handleSubmit
client/src/components/review/ReviewCard.jsx	47	ReviewCard, avatarUrl, initials, reviewer
client/src/features/auth/authSlice.js	188	authSlice, getAuthErrorMessage, getMe, initialState, loginUser, loginWithSupabase, registerUser, res, selectAuthErro
client/src/features/booking/bookingSlice.js	150	acceptBooking, bookingSlice, cancelBooking, completeBooking, createBooking, fetchBuyerBookings, fetchSellerBook
client/src/features/favorite/favoriteSlice.js	80	addFavorite, favoriteSlice, fetchFavorites, removeFavorite, res, selectFavoriteIds, selectFavoriteLoading, selectFavori
client/src/features/message/messageSlice.js	105	conv, createOrGetConversation, exists, fetchConversations, fetchMessages, messageSlice, res, selectActiveConvers
client/src/features/property/propertySlice.js	164	createProperty, deleteProperty, fetchMyProperties, fetchProperties, fetchPropertyById, idx, propertySlice, res, selectC
client/src/features/sellerVerification/sellerVerificationSlice.js	116	approveVerification, fetchAllVerifications, fetchMyVerificationStatus, rejectVerification, res, selectAllVerifications, selec
client/src/hooks/useAuth.js	35	dispatch, error, isAuthenticated, loading, role, useAuth, user, verificationStatus
client/src/hooks/useDebounce.js	12	timer, useDebounce
client/src/index.css	138	—
client/src/layouts/DashboardLayout.jsx	133	DashboardLayout, adminNav, buyerNav, handleLogout, navItems, navigate, sellerNav
client/src/layouts/PublicLayout.jsx	15	PublicLayout
client/src/lib/supabase.js	8	supabase, supabaseAnonKey, supabaseUrl
client/src/main.jsx	27	—
client/src/pages/admin/AdminBookings.jsx	144	AdminBookings, STATUSES, fetchBookings, params, res
client/src/pages/admin/AdminCommissions.jsx	221	AdminCommissions, SummaryCard, buyerTotal, commissionAmt, data, fetchCommissions, params, property, res, sel
client/src/pages/admin/AdminDashboard.jsx	261	AdminDashboard, KpiCard, fetchStats, handleApprove, handleReject, kpis, navigate, reason, res
client/src/pages/admin/AdminProperties.jsx	328	AdminProperties, LISTING_TYPES, PROPERTY_TYPES, STATUSES, fetchProperties, handleDelete, handleStatusC
client/src/pages/admin/AdminReviews.jsx	200	AdminReviews, StarRating, fetchReviews, handleDelete, params, px, res
client/src/pages/admin/UserManagement.jsx	252	ROLES, STATUSES, UserManagement, a, blob, csv, fetchUsers, handleExportCSV, handleToggleStatus, headers, p
client/src/pages/admin/VerificationManagement.jsx	327	TABS, VerificationManagement, fetchVerifications, handleApprove, handleReject, openDoc, openRejectModal, param
client/src/pages/buyer/BuyerDashboard.jsx	226	BuyerDashboard, activeCount, allBookings, completedCount, dispatch, loadData, recentBookings, reviewsCount
client/src/pages/buyer/BuyerMessages.jsx	310	BuyerMessages, conversation, conversationId, fetchConversations, fetchMessages, filteredConversations, handleSer
client/src/pages/buyer/BuyerProfile.jsx	281	BuyerProfile, currentAvatar, dispatch, fd, file, fileInputRef, handleAvatarChange, handlePasswordSave, handleProfileS
client/src/pages/buyer/Favorites.jsx	93	Favorites, dispatch, favorites, handleRemove, loading, property, result
client/src/pages/buyer/MyBookings.jsx	341	MyBookings, StatusIcon, bookings, config, dispatch, error, filterOptions, filteredBookings, getStatusBadge, handleCar
client/src/pages/buyer/MyReviews.jsx	343	MyReviews, fetchMyReviews, handleDeleteClick, handleDeleteReview, handleEditClick, handleUpdateReview, respo

File	Lines	Detected declarations
client/src/pages/public/About.jsx	218	About, features, stats, team
client/src/pages/public/Contact.jsx	310	Contact, contactInfo, faqs, handleSubmit, validationMessage
client/src/pages/public/Home.jsx	460	FeatureCard, Home, PropertySkeleton, StatCard, StepCard, fetchFeatured, handleSearch, navigate, params, res
client/src/pages/public/Login.jsx	221	Login, active, dispatch, error, exchangeGoogleSession, from, handleGoogleLogin, handleSubmit, loading, location, na
client/src/pages/public/Properties.jsx	402	CardSkeleton, Properties, SORT_OPTIONS, SidebarFilters, activeCount, debouncedFilters, dispatch, handleClearFilt
client/src/pages/public/PropertyDetails.jsx	769	BookingModal, DetailChip, ImageGallery, PropertyDetails, ReviewCard, WriteReviewModal, avg, bookings, check, da
client/src/pages/public/Register.jsx	272	Register, dispatch, error, handleSubmit, loading, navigate, password, result, strength, strengthColors, strengthLabels
client/src/pages/seller/AddProperty.jsx	523	ALLOWED_IMAGE_TYPES, AMENITIES, AddProperty, MAX_IMAGES, MAX_IMAGE_SIZE, SectionTitle, Verificatio
client/src/pages/seller/BookingRequests.jsx	360	BookingRequests, StatusIcon, config, endpoint, fetchBookingRequests, filterOptions, filteredBookings, getStatusBadg
client/src/pages/seller/CommissionSummary.jsx	338	CommissionSummary, a, blob, csvContent, data, dateRangeOptions, fetchCommissions, filterOptions, filteredCommis
client/src/pages/seller/EditProperty.jsx	609	ALLOWED_IMAGE_TYPES, AMENITIES, EditProperty, MAX_IMAGES, MAX_IMAGE_SIZE, SectionTitle, dispatch, f
client/src/pages/seller/MyProperties.jsx	256	EmptyState, MyProperties, STATUS_TABS, dispatch, getStatusBadgeClass, handleDelete, handleTabChange, loadir
client/src/pages/seller/SellerDashboard.jsx	340	SellerDashboard, StatCard, VerificationBanner, activeListings, bookLoading, bookings, dispatch, getBookingStatusBa
client/src/pages/seller/SellerMessages.jsx	298	SellerMessages, buyerId, conversation, conversationId, fetchConversations, fetchMessages, filteredConversations, ha
client/src/pages/seller/SellerProfile.jsx	367	SellerProfile, badgeMap, data, fetchProfile, fetchStats, getVerificationBadge, handleChange, handleSubmit, profile, pr
client/src/pages/seller/SellerVerification.jsx	293	ALLOWED_TYPES, FileUploadBox, MAX_FILE_SIZE, SellerVerification, StatusCard, dispatch, err, file, formData, ha
client/src/routes/AppRoutes.jsx	132	AppRoutes
client/src/routes/ProtectedRoute.jsx	20	ProtectedRoute, location
client/src/routes/RoleRoute.jsx	12	RoleRoute
client/src/services/admin.service.js	17	adminService
client/src/services/api.js	39	BASE_URL, api, token
client/src/services/auth.service.js	13	authService
client/src/services/booking.service.js	12	bookingService
client/src/services/commission.service.js	7	commissionService
client/src/services/contact.service.js	5	contactService
client/src/services/conversation.service.js	11	conversationService
client/src/services/favorite.service.js	8	favoriteService
client/src/services/property.service.js	20	propertyService
client/src/services/review.service.js	12	reviewService
client/src/services/sellerVerification.service.js	16	sellerVerificationService
client/src/SimpleTest.jsx	13	SimpleTest
client/src/store/store.js	24	store
client/src/utils/constants.js	7	BOOKING_STATUS, FURNISHING_STATUS, LISTING_TYPES, PROPERTY_STATUS, PROPERTY_TYPES, ROL
client/src/utils/formatters.js	106	ampm, calculateCommission, date, diffDay, diffHr, diffMin, diffMs, diffSec, formatDate, formatDateShort, formatPrice,
server/scripts/seed.js	440	Booking, Commission, Conversation, Favorite, Message, Property, Review, SellerVerification, User, adminUser, appr
server/src/app.js	94	adminRoutes, allowedOrigins, app, authRoutes, bookingRoutes, commissionRoutes, contactRoutes, conversationRou
server/src/config/cloudinary.js	9	cloudinary
server/src/config/db.js	15	conn, connectDB, mongoose
server/src/config/supabase.js	30	ApiError, getSupabase, supabase
server/src/controllers/admin.controller.js	187	ApiError, ApiResponse, Booking, Commission, Property, Review, SellerVerification, User, asyncHandler, getAllPrope
server/src/controllers/auth.controller.js	220	ApiError, ApiResponse, SellerVerification, User, asyncHandler, changePassword, email, existingUser, generateToker
server/src/controllers/booking.controller.js	251	ApiError, ApiResponse, Booking, Property, acceptBooking, asyncHandler, booking, cancelBooking, completeBooking
server/src/controllers/commission.controller.js	113	ApiResponse, Commission, asyncHandler, getAllCommissions, getMyBuyerCommissions, getMySellerCommissions,
server/src/controllers/contact.controller.js	23	ApiResponse, Contact, asyncHandler, contact, createContact, validateRequest
server/src/controllers/conversation.controller.js	162	ApiError, ApiResponse, Conversation, Message, Property, asyncHandler, conversation, createOrGetConversation, ge
server/src/controllers/favorite.controller.js	95	ApiError, ApiResponse, Favorite, Property, addFavorite, asyncHandler, checkFavorite, existing, favorite, getFavorites

File	Lines	Detected declarations
server/src/controllers/property.controller.js	311	ApiError, ApiResponse, Property, allowedFields, amenityList, asyncHandler, createProperty, deleteProperty, deletePr
server/src/controllers/review.controller.js	230	ApiError, ApiResponse, Booking, Property, Review, asyncHandler, createReview, deleteReview, eligibleBooking, exis
server/src/controllers/sellerVerification.controller.js	211	ApiError, ApiResponse, SellerVerification, approveVerification, asyncHandler, files, getAllVerifications, getMyVerificati
server/src/controllers/user.controller.js	109	ApiError, ApiResponse, User, allowedFields, asyncHandler, deleteUser, getAllUsers, getUserById, query, suspendUs
server/src/middleware/auth.middleware.js	52	ApiError, User, asyncHandler, authMiddleware, decoded, jwt, token, user
server/src/middleware/error.middleware.js	65	ApiError, errorMiddleware, errors, field, message, response, statusCode
server/src/middleware/rateLimit.middleware.js	27	apiLimiter, authLimiter, rateLimit
server/src/middleware/role.middleware.js	24	ApiError, authorizeRoles
server/src/middleware/upload.middleware.js	104	ApiError, documentFileFilter, handleMulterError, imageFileFilter, multer, storage, uploadPropertyImages, uploadSingl
server/src/middleware/verified.middleware.js	54	ApiError, SellerVerification, asyncHandler, isVerifiedSeller, verification
server/src/models/Booking.model.js	61	Booking, bookingSchema, mongoose
server/src/models/Commission.model.js	76	Commission, commissionSchema, mongoose
server/src/models/Contact.model.js	49	contactSchema, mongoose
server/src/models/Conversation.model.js	43	Conversation, conversationSchema, mongoose
server/src/models/Favorite.model.js	25	Favorite, favoriteSchema, mongoose
server/src/models/Message.model.js	36	Message, messageSchema, mongoose
server/src/models/Property.model.js	147	Property, imageSchema, mongoose, propertySchema
server/src/models/Review.model.js	45	Review, mongoose, reviewSchema
server/src/models/SellerVerification.model.js	62	SellerVerification, documentSchema, mongoose, sellerVerificationSchema
server/src/models/User.model.js	83	User, bcrypt, mongoose, obj, userSchema
server/src/routes/admin.routes.js	21	authMiddleware, authorizeRoles, express, router
server/src/routes/auth.routes.js	27	authMiddleware, express, router
server/src/routes/booking.routes.js	34	authMiddleware, authorizeRoles, express, router
server/src/routes/commission.routes.js	18	authMiddleware, authorizeRoles, express, router
server/src/routes/contact.routes.js	9	express, router
server/src/routes/conversation.routes.js	19	authMiddleware, express, router
server/src/routes/favorite.routes.js	21	authMiddleware, authorizeRoles, express, router
server/src/routes/property.routes.js	64	authMiddleware, authorizeRoles, express, isVerifiedSeller, router
server/src/routes/review.routes.js	30	authMiddleware, authorizeRoles, express, router
server/src/routes/sellerVerification.routes.js	33	authMiddleware, authorizeRoles, express, router
server/src/routes/user.routes.js	22	authMiddleware, authorizeRoles, express, router
server/src/server.js	50	PORT, app, connectDB, missingEnvVars, requiredEnvVars, server, startServer
server/src/services/cloudinary.service.js	89	cloudinary, deleteFromCloudinary, result, streamifier, uploadAvatar, uploadPropertyImage, uploadStream, uploadToC
server/src/services/commission.service.js	90	Commission, buyerCommissionAmount, commission, createSellerCommission, getCommissionSummary, recordBuye
server/src/services/review.service.js	34	Property, Review, stats, updatePropertyRating
server/src/utils/ApiError.js	17	ApiError
server/src/utils/ApiResponse.js	10	ApiResponse
server/src/utils/asyncHandler.js	9	asyncHandler
server/src/utils/authLogger.js	25	authLogPath, entry, fs, logAuthEvent, logDirectory, path
server/src/utils/constants.js	99	ALLOWED_DOCUMENT_TYPES, ALLOWED_IMAGE_TYPES, BOOKING_STATUS, COMMISSION_RATES, COM
server/src/utils/pagination.js	27	buildPaginationMeta, getPagination, limit, page, skip, totalPages
server/src/utils/validateRequest.js	18	ApiError, errors, formattedErrors, validateRequest
server/src/validators/auth.validator.js	70	changePasswordValidator, loginValidator, registerValidator
server/src/validators/booking.validator.js	31	bookingValidator, date, today
server/src/validators/contact.validator.js	26	contactValidator
server/src/validators/message.validator.js	16	conversationValidator, messageValidator

File	Lines	Detected declarations
server/src/validators/property.validator.js	85	propertyValidator, updatePropertyValidator
server/src/validators/review.validator.js	18	reviewValidator

14. Suggested Mentor Presentation

Present the project in this order: (1) problem and broker-free marketplace goal, (2) system architecture, (3) user roles and protected routes, (4) authentication including the Supabase-to-MongoDB handoff, (5) property/booking/messaging/review workflows, (6) database model relationships, (7) security controls, (8) deployment dependencies, and (9) current limitations and next steps.

One-sentence explanation: HaikyuuHeights is a role-based MERN real-estate marketplace where React provides the buyer, seller, and admin experience; Express exposes protected REST APIs; MongoDB stores business data; Cloudinary stores media; and Supabase validates Google identity before the backend issues its own JWT.

End of document